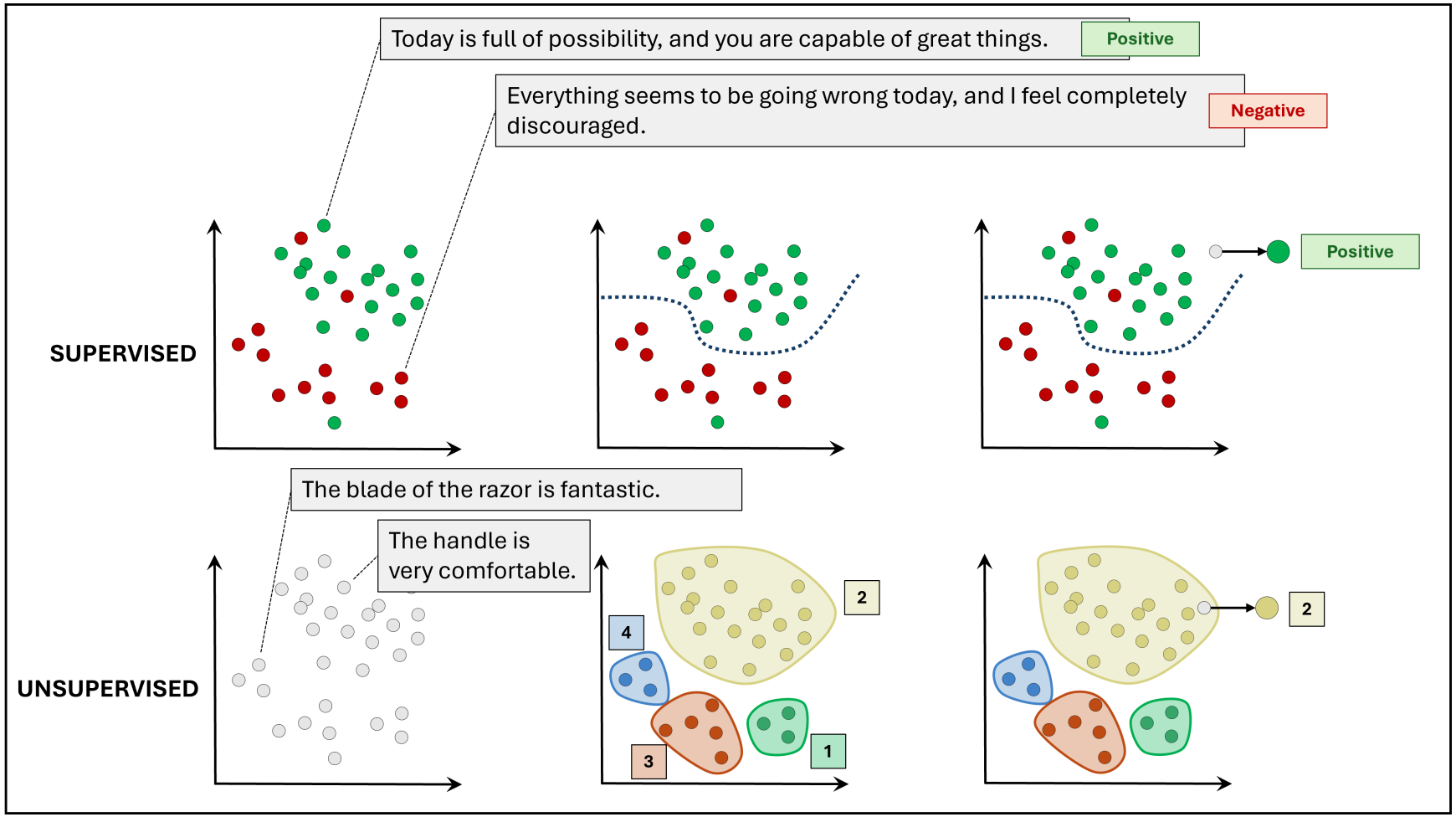
Text Modelling

Filippo Chiarello & Vito Giordano

Natural Language Processing Tasks

- **Supervised tasks**: A supervised NLP task is a task in which each text input is associated with a *ground truth*, that is, an expected output, reference label, or human annotation. The objective is to produce outputs consistent with the available ground truth for new inputs.
- **Unsupervised tasks**: An unsupervised NLP task is a task in which text data are available *without ground truth*, meaning that no expected output or reference annotation is provided for each instance. The objective is to identify patterns, structures, groups, or regularities in the text.

Natural Language Processing Tasks



Natural Language Processing Tasks

Supervised Tasks:

- Document Classification
- Sentence Classification
- Named Entity Recognition
- Relation Extraction
- Sentiment Analysis

Unsupervised Tasks:

- Topic Modelling
- Clustering
- Text Similarity

Dataset

First we have to select a dataset.

We selected a **dataset of tweets** from Kaggle that captures users' perceptions of ChatGPT.

📦 Source:
The ChatGPT Tweets Dataset

For our analysis, we selected a sample of **1,000 tweets**.

Dataset

This dataset contains the tweets which talks about how ChatGPT can be used for different tasks.

Twitter Data Collection

Collect tweets talking about ChatGPT

Natural Language Processing

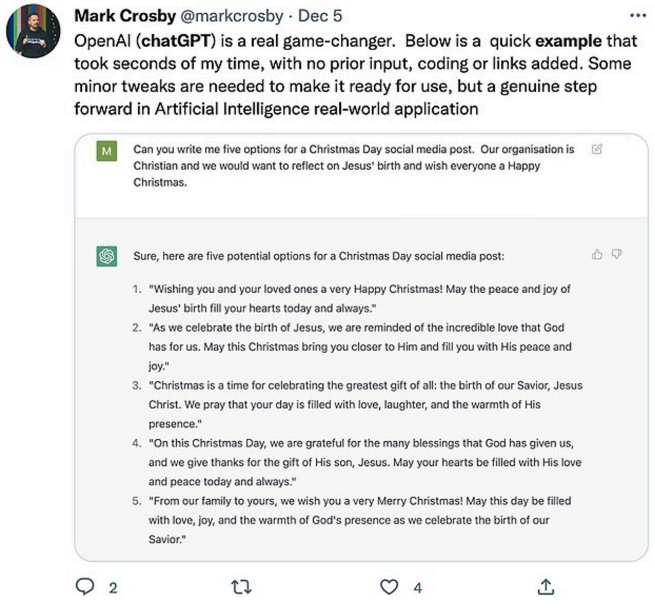
Use NLP to process the tweets collected

Task Extraction

Extract Task asked by Twitter users to ChatGPT

QUERY: keywords ***chatgpt*** or ***gpt***

The collection through Twitter API brought to the generation of a dataset containing **3 mln** tweets.



Dataset

We can see here the method applied for extracting the ChatGPT tasks.

Twitter Data Collection

Collect tweets talking about ChatGPT

Natural Language Processing

Use NLP to process the tweets collected

Task Extraction

Extract Task asked by Twitter users to ChatGPT

Technology	Connection Element	Action Verb	Object
gpt, (gpt), #gpt, (#gpt), gpt3, (gpt3), #gpt3, (#gpt3), gpt-3, (gpt-3), gpt 3, (gpt 3), #gpt-3, (#gpt-3), chatgpt …	to, explains, can, could, about, for, may, might, knows how, knows how to, how to, can beused in, can be can support in, can support, can support at, how to help in, how to help for, how to help at, how to support in, how to support for	Optional Adverb + Verb + Optional word + Optional Article + Optional Possessive Adjective + Optional Adjective + Optional Compound	Direct Object (**excluding**: me, you, it, him, her, us, them, this, that, those, these)

I **asked chatgpt to write a code for website**, it's amazing. ChatGPT is the perfect tool to help you get creative ideas and automate conversations quickly and accurately. #ChatGPT #conversationalAI #chatbots

Dataset

And also here some results of the tasks extraction.

Twitter Data Collection

Collect tweets talking about ChatGPT

Natural Language Processing

Use NLP to process the tweets collected

Task Extraction

Extract Task asked by Twitter users to ChatGPT

Top-20 tasks for number of tweets.

Task	# of tweets	Task	# of tweets
Write a poem	887	Gives answers	269
Write code	775	Write a script	234
Write an essay	617	Write tool	225
Write a song	509	Passed a wharton mba exam	220
Answer questions	477	Affect web3 space	212
Write story	463	Do work	196
Write article	332	Generate content	184
Write a tweet	326	Mean for the future	174
Generate code	301	Gave the correct answer	161
Generate text	296	Write emails	159

9,558 **tweets** have at least one task.

5,554 different **tasks** are collected.

Dataset

We now import the dataset.

```
#{r}
# Import packages
library(tidyverse)

# Import tweet
tweet_data <- read_csv("data/sample_tweet.csv")
```

```
## # A tibble: 4 × 16
##   tweet_id original_text       sentiment tasks users technologies
##      <dbl> <chr>               <chr>     <chr> <chr> <chr>
## 1  1.60e18 "[GPT-3] This post… neutral   ensu… <NA>  artificial …
## 2  1.60e18 "Today is the pre-… neutral   unde… <NA>  database
## 3  1.60e18 "@DeclanMccreary T… negative  inve… acad… model
## 4  1.60e18 "Screenshot of a p… neutral   wrot… <NA>  model
## # ℹ 10 more variables: organizations <chr>,
## #   job_competencies <chr>, job_profiles <chr>, nouns <chr>,
## #   verbs <chr>, adjectives <chr>, subjects <chr>,
## #   objects <chr>, predicates <chr>, topic <chr>
```

Dataset

Let's select only useful information.

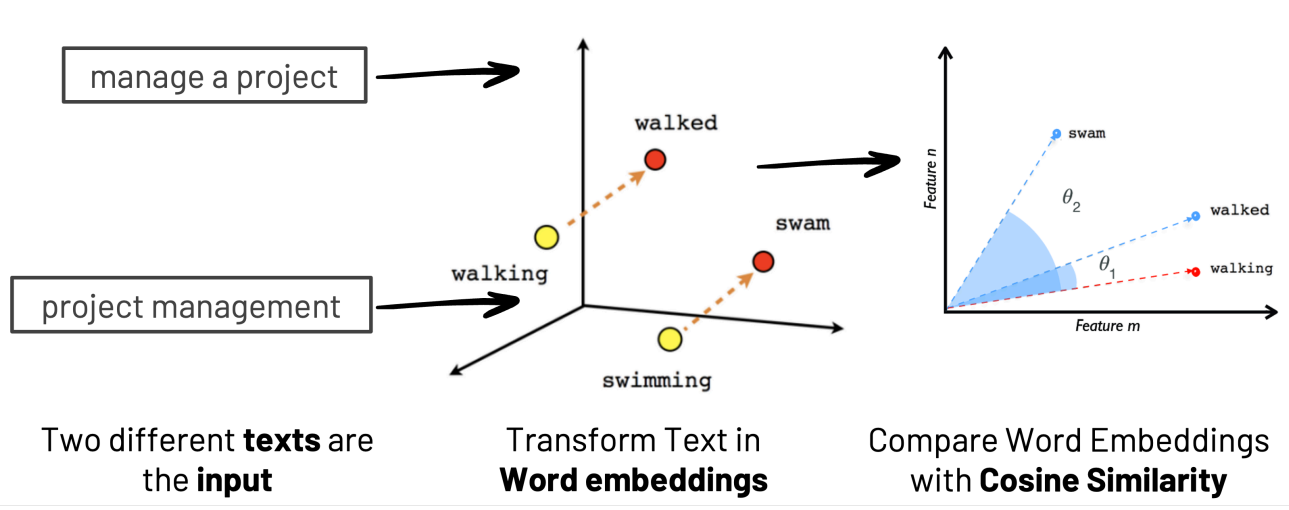
```
#{r}
# Select only useful information
tweet_data <- tibble(tweet_data) |>
  select(tweet_id, original_text, tasks)
```

```
## # A tibble: 6 × 3
##   tweet_id original_text                                        tasks
##      <dbl> <chr>                                                <chr>
## 1  1.60e18 "[GPT-3] This post discusses the need for an au… ensu…
## 2  1.60e18 "Today is the pre-fine-tuning day! \r\n\r\nI'm … unde…
## 3  1.60e18 "@DeclanMccreary They don't program it to inven… inve…
## 4  1.60e18 "Screenshot of a project I'm working on using t… wrot…
## 5  1.60e18 "@Vkumzy @MsRabbitKick @rippleitinNZ It's OpenA… writ…
## 6  1.60e18 "@Andrew___Baker When I tried it, GPT-3 nailed … nail…
```

Semantic similarity

Semantic similarity

Semantic similarity **measures how close in meaning two pieces of text are**. To do this, the model converts each text into a numerical representation called a **word embedding** — a vector that captures the meaning of the text in context.



We will use LLM with **Transformers** architecture for obtaining the word embeddings of the text. For calculating Semantic similarity, we will use **cosine similarity**.

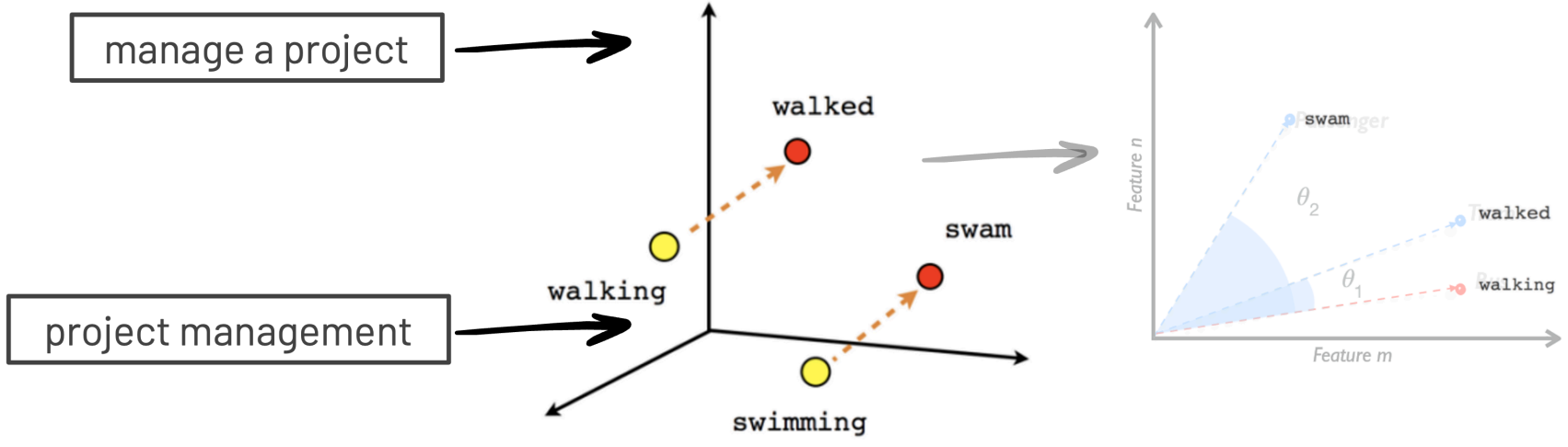
Semantic similarity

In our case, we want to **evaluate the similarity between the tasks identified in our tweet dataset**.

Task	Skill	BERT similarity
Compose music	Compose music	1.00
Write English	Write English	1.00
Write caption	Write captions	0.99
Provide information	Provide information	0.96
Write job description	Write job descriptions	0.96
Write scientific paper	Write scientific publications	0.87
Do tax	Calculate tax	0.87
Generate weight loss plan	Develop weight loss schedule	0.83
Write love song	Write songs	0.80
Do you homework	Assist children with homework	0.76

Semantic similarity

In our case, we want to **evaluate the similarity between the tasks identified in our tweet dataset**. Initially we have to identify the word embeddings of the task extracted.



Two different **texts** are the **input**

Transform Text in **Word embeddings**

Compare Word Embeddings with **Cosine Similarity**

Semantic similarity

The first step is create a dataset with only the tasks.

```
#{r}
# Create a dataset with only the tasks
task_data <- tweet_data |>
  select(tasks) |>
  unique()
```

```
## # A tibble: 6 × 1
##   tasks
##   <chr>
## 1 ensure safety
## 2 understand a simple mysql database
## 3 invent citations, it can construct language
## 4 wrote a paragraph
## 5 write an excuse letter
## 6 nailed a number
```

Semantic similarity

Obtained the tasks, for evaluating the similarity we have to extract the word embedding of each tasks.

For doing this we have to use Python packages **transformers** and **torch**. To do it, you need a Python environment that can be called from R through the package **reticulate**.

As Python environment, we suggest to install Anaconda3.

Semantic similarity

Once Anaconda is installed, you should also install the libraries **transformers** and **torch** through the Anaconda Prompt:

```
pip install pytorch
pip install transformers
```

Then, install reticulate in RStudio.

```
#{r}
#install.packages("reticulate")
```

Semantic similarity

Set the python environment.

```
#{r}
library(reticulate)

# Use the desired Python environment (adjust the path to your specific environment)
use_python("C:/Users/Vito Giordano/anaconda3/python.exe", required = TRUE)
py_config()
```

```
## python:         C:/Users/Vito Giordano/anaconda3/python.exe
## libpython:      C:/Users/Vito Giordano/anaconda3/python311.dll
## pythonhome:     C:/Users/Vito Giordano/anaconda3
## version:        3.11.4 | packaged by Anaconda, Inc. | (main, Jul  5 2023, 13:38:37) [MSC v.1916 64 bit
## Architecture:   64bit
## numpy:          C:/Users/Vito Giordano/anaconda3/Lib/site-packages/numpy
## numpy_version:  1.24.3
##
## NOTE: Python version was forced by use_python() function
```

Semantic similarity

Set the python environment.

```
#{r}
# Import transformers library
transformers <- import("transformers")
torch <- import("torch")
```

Transformers is a library developed by Hugging Face that provides pre-trained models and tools for NLP.
PyTorch is a machine learning library developed by Facebook that provides tools for deep learning and tensor computation.

Semantic similarity

Once the Python environment is set, we create a function for extracting the word embeddings from the tasks using a model:

```
#{python}
from transformers import AutoTokenizer, AutoModel
import torch

# Select a pretrained model and tokenizer
model_name = "bert-large-uncased"

# Initialize the tokenizer
tokenizer = AutoTokenizer.from_pretrained(model_name)

# Initialize the model
model = AutoModel.from_pretrained(model_name)
```

Semantic similarity

Once the Python environment is set, we create a function for extracting the word embeddings from the tasks using a model:

```
#{python}
import numpy as np

# Function to compute sentence embedding (mean of the token embeddings)
def get_embedding(text):

  # tokenize the input text
  inputs = tokenizer(text, return_tensors="pt", padding=True, truncation=True)

  with torch.no_grad():

      # obtaining the word embeddings
      outputs = model(**inputs)

  # embedding mean of the tokens
  return outputs.last_hidden_state.mean(dim=1).numpy()[0]
```

Semantic similarity

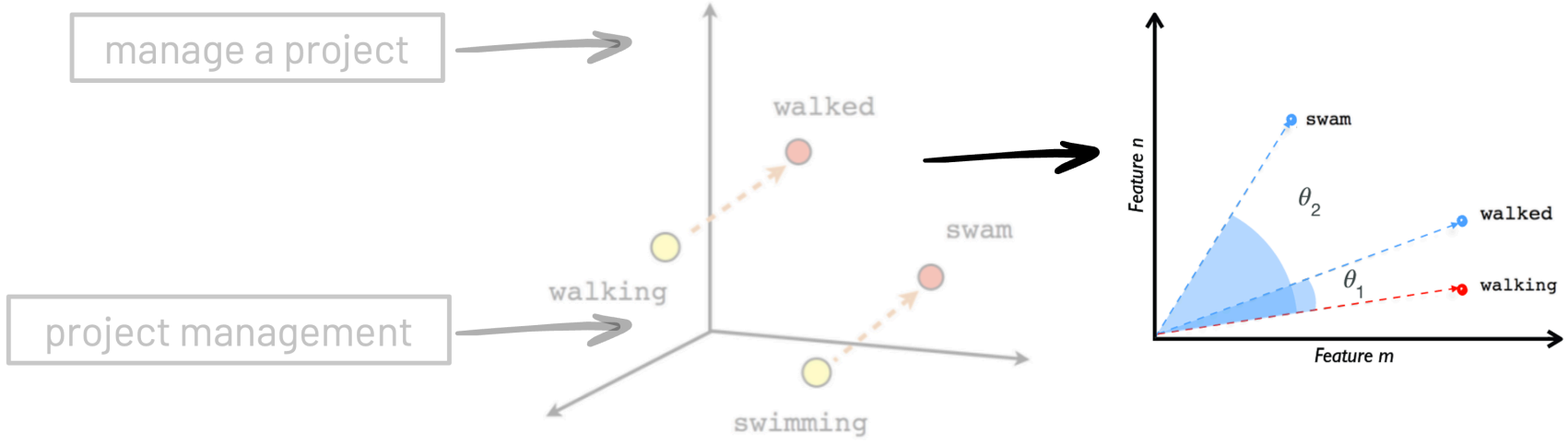
Now we calculate the word embeddings of the tasks.

```
#{python}
# Convert the R datasets in python
task_data = r.task_data

# Identify word embeddings
task_data['embedding'] = task_data['tasks'].apply(get_embedding)
```

Semantic similarity

Obtained the word embeddings of the tasks, we can calculate the cosine similarity between them.



Two different **texts** are the **input**

Transform Text in **Word embeddings**

Compare Word Embeddings with **Cosine Similarity**

Semantic similarity

First, we have to create a dataset with all the combinations of tasks and their embeddings.

```
#{python}
from itertools import combinations
import pandas as pd

# Create list unique task combinations
task_pairs = list(combinations(task_data.itertuples(index=False), 2))

# Create the task combinations dataset
rows = []
for task1, task2 in task_pairs:
    rows.append({
        'task1': task1.tasks,
        'embedding1': task1.embedding,
        'task2': task2.tasks,
        'embedding2': task2.embedding,
    })

# Convert the dataset in Dataframe
task_combinations = pd.DataFrame(rows)
```

Semantic similarity

Now we can calculate the cosine similarity.

```
#{python}
from sklearn.metrics.pairwise import cosine_similarity

# Calculate the cosine similarity between tasks
task_combinations['cosine_similarity'] = task_combinations.apply(
  lambda row: cosine_similarity([row['embedding1']], [row['embedding2']]).item(),axis=1)

# Select only the task cosine similarities
task_combinations = task_combinations[['task1', 'task2', 'cosine_similarity']]
```

Semantic similarity

Here we can see the results of cosine similarity.

```
#{r}
# Convert the python dataset in R
task_similarities <- tibble(py$task_combinations)
```

```
## # A tibble: 6 × 3
##   task1                  task2                cosine_similarity
##   <chr>                  <chr>                            <dbl>
## 1 passes the turing test passed the turing te…             0.97
## 2 write a blog post      create a blog post                0.97
## 3 writes a seinfeld scene write a seinfeld sce…            0.96
## 4 passes the turing test fails a basic turing…             0.96
## 5 write poetry           write poems                       0.96
## 6 write a haiku          write some haiku                  0.95
```